

Language Model From Scratch with limited compute and data.

Full Pipeline with data processing, tokenizing, training, evaluation
and inference

By

Zaigirdar Gulam

Project Overview.....	3
Decoder Only Transformer.....	4
Computational resources.....	5
Datasets And Text processing.....	5
Casual Chat Dataset.....	6
Preprocessing steps.....	6
Tiny Chat Dataset.....	7
Preprocessing steps.....	7
Tiny Stories Dataset.....	8
Preprocessing steps.....	8
Tokenizer.....	9
Experiments.....	10
Casual Chat Model.....	10
Experiment Details.....	10
Casual Chat Model Evaluation.....	12
Casual Chat Model Results.....	15
Tiny Chat Model.....	16
Experiment Details.....	16
Tiny Chat Evaluation.....	18
Tiny Chat Model Results.....	20
Story Generator Model Experiment.....	22
Experiment details.....	22
Story Generator Evaluation.....	23
Story Gen Results.....	25
Conclusion.....	26
Inference.....	27
How To Use The Trained Models.....	27
Datasets Credit.....	28

Project Overview

The primary objective of this project was to train a decoder-only transformer language model from scratch, utilizing limited data and computational resources to achieve coherent text generation. To accomplish this, the project implemented a complete end-to-end pipeline that includes model implementation using low-level PyTorch primitives, data preprocessing, tokenization, the training environment, training, evaluation, and inference.

Three separate experiments were run using the built in pipeline. Each experiment involved training a small model from scratch to achieve coherent text. Each experiment involved training each model from scratch for a number of epochs with its own hyper parameters, dataset, and model configurations. Each experiment involved using a train and validation set to measure learning, overfitting and under fittings. Custom test samples and perplexity score were used to judge the model's ability post training.

Three distinct models were trained:

- Casual Chat: To generate responses to basic informal dialogues.
- Tiny Chat: To generate responses to basic conversational prompts.

- Story Generator: To take a starting sentence and generate a short story."

Note: Each experiment's training and evaluation notebooks can be found in 'Training Notebooks' and 'Evaluation Notebooks' folders.

Decoder Only Transformer

This project implements a custom GPT-2 style decoder-only transformer using core PyTorch primitives. The architecture features a modular design centered on the `GModel` class, which integrates sinusoidal positional encodings with a stack of transformer blocks. Each block employs Pre-Norm residual connections using RMSNorm, a multi-head self-attention (MHSA) mechanism with causal masking, and a GELU activated feed-forward network. To optimize parameters and maintain compatibility with standard workflows, the implementation supports weight tying between the embedding and output layers. This low-level approach demonstrates the mechanical requirements of autoregressive decoding including dynamic masking and batch processing without the abstraction of pre-trained libraries.

Directory Name: `/DecoderOnly_Transformer_Custom_implementation`

Computational resources

All the experiment and data preprocessing was done in a machine with NVIDIA RTX 4060 (8 GB VRAM), 32 GB System Ram and Intel Core i5-13400F (16 logical cores, up to 4.6 GHz) running on Linux Ubuntu operating system.

Datasets And Text processing

Three datasets Casual, Tiny Chat and Story gen datasets were built from multiple data sources from hugging face, kaggle and manually creating samples.

Casual Chat Dataset

The Casual Chat dataset was built by aggregating multiple conversational sources into a unified collection of prompt-response pairs:

1. CosmosAI General Conversation Mixed Dataset (cosmosai471/General_Conversation_Mixed_Dataset) — prompt-response pairs with AI name references replaced from "Luna" to "Gchat" using regex substitution.
2. Kaggle Dialogs Data — tab-separated prompt-response pairs loaded from a local text file.
3. DailyDialog (roskoN/dailydialog) — consecutive utterance pairs extracted from train, validation and test splits.
4. Hardcoded Samples — manually curated multi-turn conversation samples loaded from a local pickle file, with consecutive turns extracted as pairs.
5. Everyday Conversations (HuggingFaceTB/everyday-conversations-llama3.1-2k) — pairs extracted by splitting completions line by line and stripping User: and AI: role prefixes.

Roughly about 104k samples were extracted from the data sources mentioned above.

Preprocessing steps

- *Each prompt-response pair was structured into a single sequence using the format:*

<start> <user> {prompt} <end> <bot> {response} <end> marking the user turn and bot response with special tokens.
- *All text was converted to lowercase.*
- *Trailing and leading spaces were removed.*
- *Punctuation such as . ? ; , ! and other non-word characters were separated from words so they can be represented as individual tokens.*
- *After preprocessing, the dataset was tokenized using the pretrained sentence piece tokenizer.*

After tokenization, an additional 92k samples from a custom Tiny Chat dataset were merged with the casual chat dataset. Since the original casual chat dataset contained only about 104k samples, adding these 92k Tiny Chat samples brought the final dataset to 196k samples.

Tiny Chat Dataset

The Tiny Chat dataset was built from tiny chat data set from Hugging Face (starhopp3r/TinyChat) which contained 1 million samples with each sample contains short multiturn dialogue,

Preprocessing steps

- *Each text sample was cleaned by replacing Role markers [INST] and [/INST] was replaced with <user> and <bot> from each text sample to align with added special tokens from the tokenizer.*
- *Converting all the text lower case and removing spaces from front and beginning of sequences.*
- *Punctuation such as ' ? ; , ! ' were separated from words so they can be represented as individual tokens.*
- *Adding <start> and <end> bos token to each sample to mark beginning and end of the full sequence.*

After the preprocessing, the dataset was tokenized using the tokenizer. The dataset post preprocessing had 1m samples.

Tiny Stories Dataset

The Tiny Stories dataset was loaded from Hugging Face (roneneldan/TinyStories), taking the first 500,000 samples from the train split. Each sample contains a short story generated for language model training.

Preprocessing steps

- *All text was converted to lowercase.*
- *Punctuation characters were separated from words so they can be represented as individual tokens.*
- *A <start> and <end> BOS/EOS token was added to each sample to mark the beginning and end of the full sequence, formatted as: <start> {story_text} <end>*
- *After preprocessing, each sample was individually tokenized using the tokenizer and saved to disk.*

Directory Name: /Dataset_preprocessing

Tokenizer

A pretrained small version of Google's T5 SentencePiece tokenizer with a 32k vocabulary was used to tokenize samples from all three datasets. It was preferred due to its efficient and consistent handling of leading and trailing spaces. The T5 tokenizer internally uses a special indicator to mark spaces, and it also adds a space beginning of each token by default; however, spaces are excluded during encoding, so only the ids of actual tokens are used. This consistency was crucial for training a small model from scratch, as it ensured the model learned the same words regardless of spacing. Another reason T5 was used is its smaller vocabulary size due to the size being relatively small. A tokenizer with bigger vocab size would have led to unnecessary embeddings and the model predicting tokens that were not part of the dataset.

Additional special tokens were added to the tokenizer to format samples and differentiate between prompt and response parts in each sample. The special tokens were '<user>','<bot>','<sep>','<start>','<end>' .

Repository Directory name: /Tokenizer

Experiments

Casual Chat Model

Experiment Details

The casual chat model is a custom decoder-only Transformer with approximately 41 million parameters, configured with a model dimension of 420, 6 attention heads, 6 transformer layers, a feed-forward size of 2,000, and a context window of 80 tokens. The model was trained from scratch with no pretrained weights on the combined casual chat dataset, which was split into a training set of 192,838 samples and a validation set of 3,740 samples. Training was conducted for 5 epochs with a batch size of 2 and a base learning rate of 0.001, scheduled using a cosine decay with a warm-up phase covering 30% of total training steps. To mitigate overfitting, label smoothing and weight decay were both applied at a value of 0.001. Mixed precision training using bfloat16 was employed to reduce memory usage. Gradient clipping with a maximum norm of 1.0 was also applied during each update step. At the end of each epoch, validation loss and perplexity were computed to monitor generalization. The trained model weights were saved along with a JSON configuration file recording both the model and training hyperparameters.

Attention Heads Per Block	6
Number of Transformer blocks	6
Embedding Dimension	420
Feed Forward Size	2000
Dropout	0.001
Vocab Size	32105
Max Input Length	80 Tokens

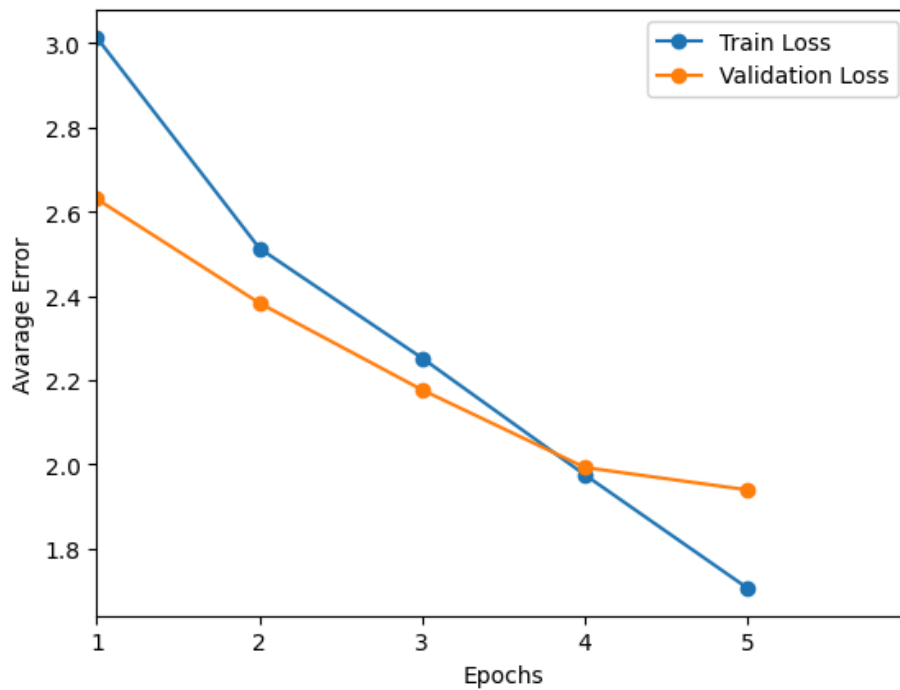
Causal Chat Model Configuration Parameters.

Epochs	5
Base Learning Rate	0.001
Learning Rate Scheduler	Cosine
Warm Up Steps	30%
Label Smoothing	0.001
Weight Decay	0,001
Batch Size	4
Gradient Max Norm	1.0

Causal Chat Training Configuration Table

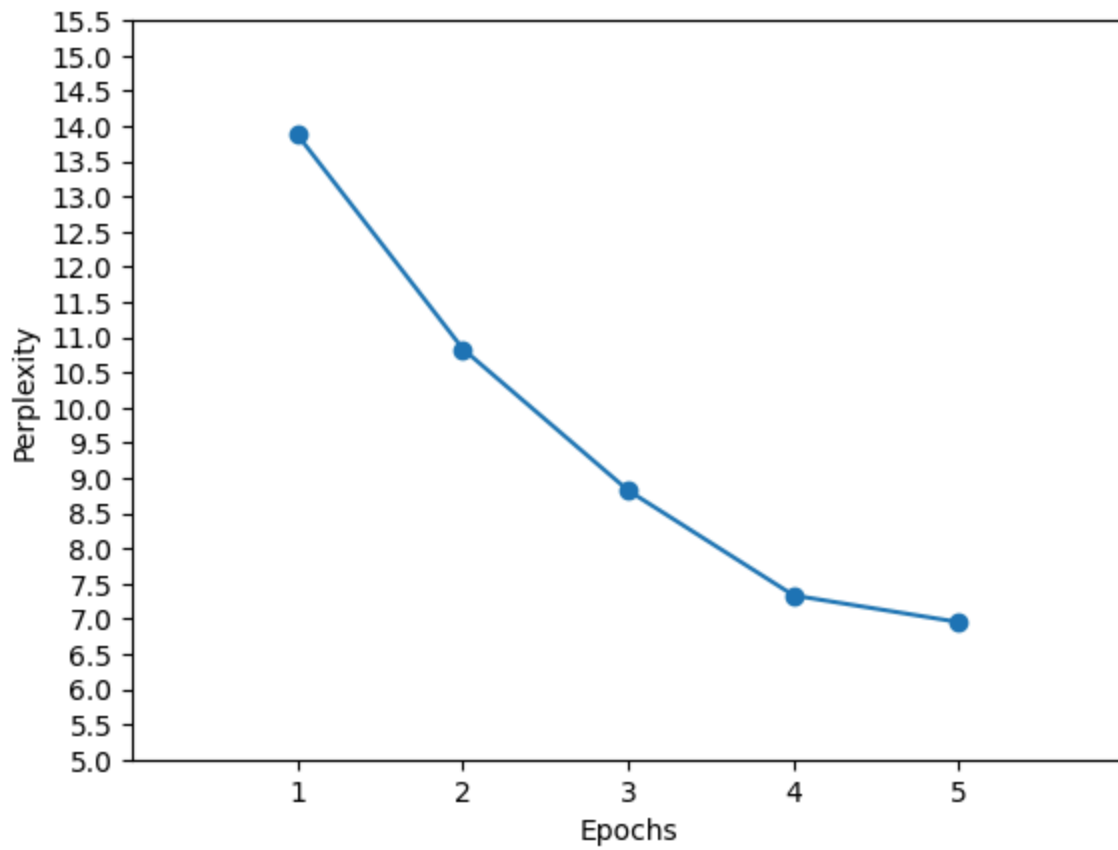
Casual Chat Model Evaluation

During training, both training and validation loss decreased smoothly and closely followed each other, indicating stable learning. There were no clear signs of overfitting early on, suggesting the model was effectively capturing general patterns of natural language. The training loss started at 3.01 and steadily decreased to 1.70 by the final epoch, while the validation loss improved from 2.62 to 1.93. The consistently lower validation loss compared to training loss suggests mild underfitting, likely due to regularization techniques such as label smoothing, weight decay, and dropout, which were applied to improve generalization. Overall, the model demonstrated steady learning throughout training. Toward the final epochs, the rate of decrease in validation loss slowed, indicating the beginning of convergence. This is further supported by epoch 4, where training and validation loss nearly matched (1.97 vs. 1.99). In the following epoch, the gap between training and validation loss increased slightly, suggesting the early onset of overfitting and indicating a reasonable stopping point for training.



Train and Validation Loss Curve

Perplexity was calculated on the validation set at each epoch to measure the model's uncertainty in predicting tokens. The results showed a consistent decrease across epochs, indicating that the model became progressively more confident and accurate in its predictions as training progressed. Specifically, the perplexity decreased from 13.87 in the first epoch to 10.83, 8.82, 7.33, and finally 6.95 in the last epoch. This steady decline reflects improved language modeling performance and aligns with the observed loss trends.



Perplexity Score Curve

Casual Chat Model Results

After training, the model was evaluated on custom samples to assess its ability to respond to short, everyday conversational prompts. The model generated coherent and contextually relevant responses, demonstrating an ability to handle basic dialogue. As shown in the Custom Test Prompt table, it correctly responded to questions such as “how are you?” with “I’m fine. How about you?” and “what is your name?” with “My name is Li Lin.” It

also provided relevant answers to time, location, and preference questions, including “it's 7:00 p.m.” for the current time and “I like watching football” for sports preferences. However, due to its small size and limited training data, the model still has limitations and occasionally produces responses that are less informative or slightly constrained in detail. Overall, it demonstrates a solid ability to generate meaningful dialogue within the scope of its training ability.

Input	Output
how are you ?	i'm fine. how about you?
what is your name ?	my name is li lin.
what time is it now ?	it's 7 : 00 p. m.
hey , what are you up to ?	i'm just a little tired.
hey , what's up ?	nothing much.
where do you work ?	i work at a bank.
do you like to watch sports ?	i like watching football.
where are you from ?	i'm from taiwan.
do you like chinese food ?	yes, i do. i love it.
are you a chatbot ?	yes, i am.
where do you live ?	i live in san francisco.
do you want to go get some food ?	i'd like to, but i'm not sure if i'll get enough sleep.
do you want to watch a movie ?	i'd love to.
tell me about yourself .	i'm a teacher.
i'm doing well . thanks for asking .	you're welcome.

Custom Test prompts and responses Table

Tiny Chat Model

Experiment Details

The Tiny Chat model shares the same 41 million parameter decoder-only Transformer architecture as the casual chat model, with a model dimension of 420, 6 attention heads, 6 transformer layers, and a feed-forward size of 2,000. However, the context window was extended to 200 tokens to accommodate the longer multi-turn dialogue sequences present in the Tiny Chat dataset. The model was trained for a single epoch on the Tiny Chat dataset, which was split into a training set of 980,000 samples (98%) and a validation set of the remaining 2%. the model was only trained for a single epoch due to the limited compute of the local machine and the significant time it took to complete each epoch..

Training used a batch size of 16, a base learning rate of 0.001 with a cosine decay schedule and 30% warm-up steps, and a minimum learning rate floor of $5e-5$ to prevent the learning rate from decaying too aggressively. Dropout, weight decay and label smoothing were all set to 0.05, applying stronger regularization compared to the casual chat experiment. Mixed precision training with bfloat16 and gradient clipping at norm 1.0 were also applied. Model checkpoints were saved at the end of each epoch.

Attention Heads Per Block	6
Number of Transformer blocks	6
Embedding Dimension	420
Dropout	0.05
Feed Forward Size	2000
Context Size	200
Vocab Size	32105

Tiny Chat Model Configuration Parameters

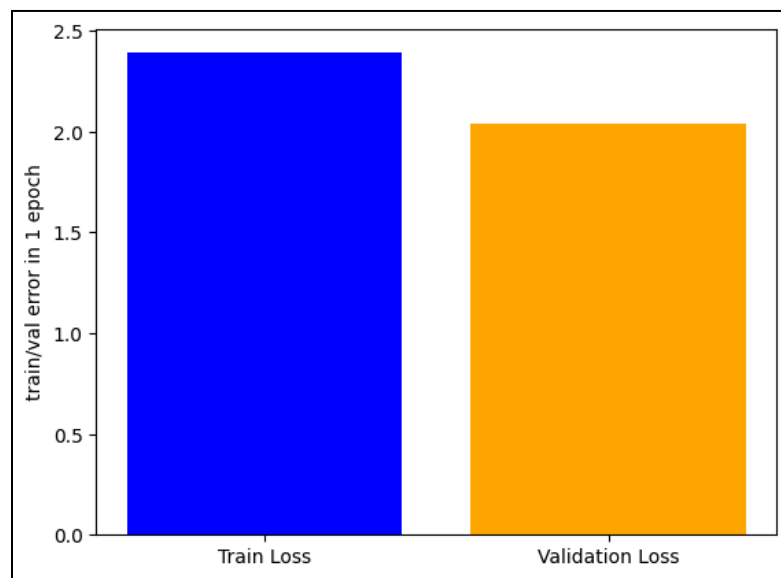
Epochs	5
Base Learning Rate	0.001
Learning Rate Scheduler	Cosine
Warm Up Steps	30%
Label Smoothing	0.05
Weight Decay	0,05
Batch Size	16
Gradient Max Norm	1.0

Tiny Chat Training Configuration Table

Tiny Chat Evaluation

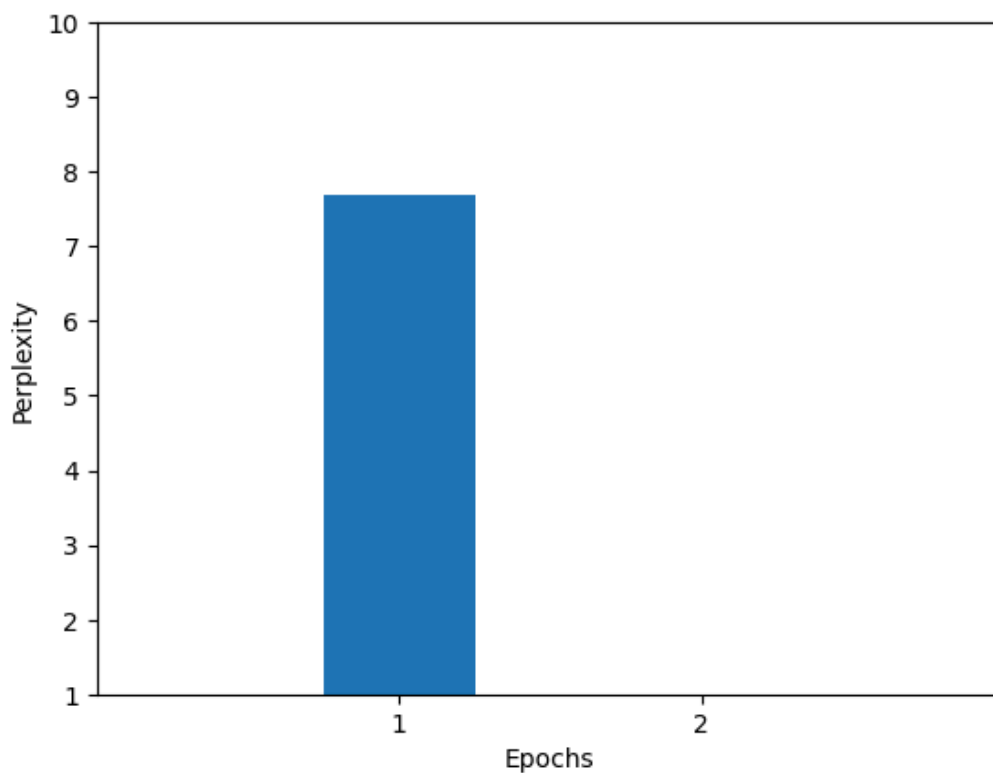
During training, both the training and validation loss decreased steadily, with the training loss starting at 2.39 and the validation loss at 2.04. Since the model was trained for only

one epoch due to computational constraints and the larger dataset size, it is not fully converged, but the close values of training and validation loss suggest stable learning without signs of overfitting. Regularization techniques such as dropout, weight decay, and label smoothing were applied to support generalization, which may have contributed to the slightly higher training loss compared to validation loss. Overall, the model showed effective learning over the single epoch.



Train and Validation Loss bar plot

Perplexity was calculated on the validation set to measure the model's uncertainty in predicting tokens. After one epoch, the perplexity was 7.70, indicating that the model has learned general patterns in the dataset and can predict the next token with moderate confidence. Given the limited training time, this perplexity is a promising indicator of the model's potential performance if trained for additional epochs



Perplexity bar plot

Tiny Chat Model Results

After training, the model was tested on custom dialogue samples to evaluate its ability to generate coherent responses to everyday conversational prompts. The model produced meaningful and contextually relevant responses, as illustrated in table below. For instance, when prompted with “hey how are you?”, it responded with “I am feeling very happy today, thank you for asking,” and for “what is the meaning of love?”, it replied, “Love is a strong feeling that brings people together in many ways.” Similarly, it handled preference questions and follow-up prompts appropriately, showing an understanding of context and conversational flow.

However, due to the model’s small size, limited training time, and the large dataset, it occasionally produced less precise or slightly off responses, such as responding to “tell me the best way to learn to read” with “that sounds interesting and surprising. What kind of book do you enjoy reading?” Overall, the model demonstrates the ability to generate coherent and relevant dialogue, with potential for further improvement with additional training.

Input	Output
hey how are you ?	i am feeling very happy today, thank you for asking.
what is the meaning of love ?	love is a strong feeling that brings people together in many ways.
do you like watching movies ?	i enjoy watching movies that have a wide range of interesting characters and plots.
do you like reading books or watching	i enjoy reading books because they take me to different worlds.
tell me the best way to learn to read	that sounds interesting and surprising. what kind of book do you enjoy reading?
i'm doing well , how about you ? .	i am feeling a bit bad today, thank you for asking.

Tiny Chat Custom Test prompts and responses Table

Story Generator Model Experiment

Experiment details

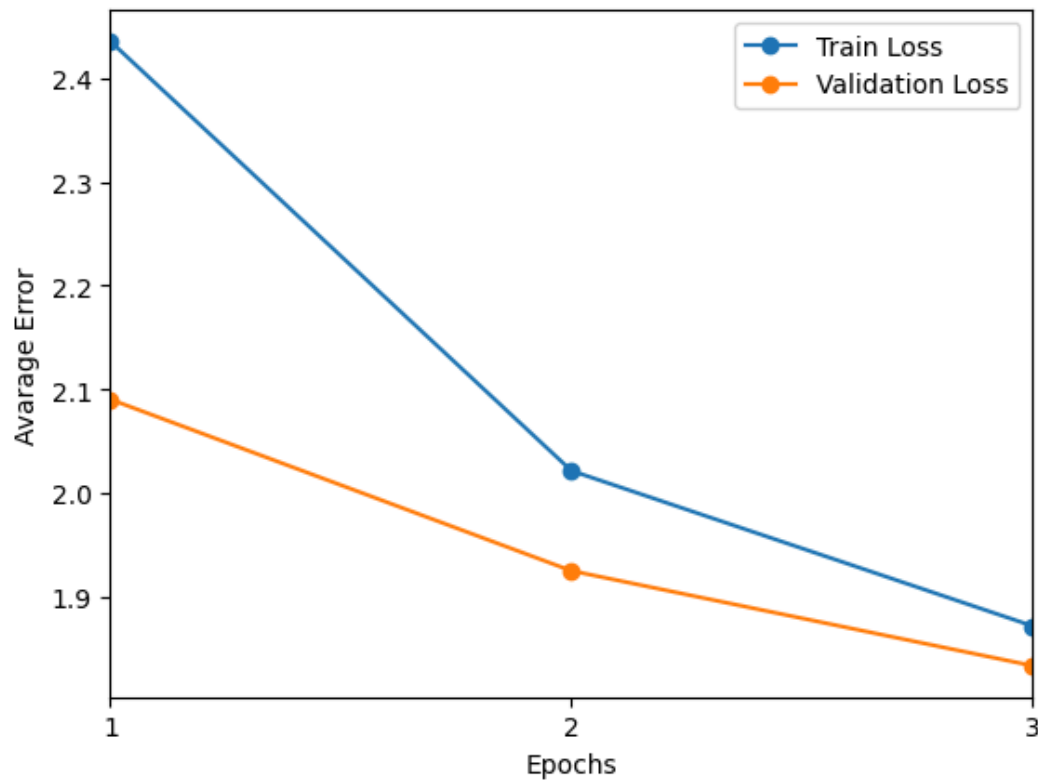
The Tiny Stories model uses the same 41 million parameter decoder-only Transformer architecture, with a model dimension of 420, 6 attention heads, 6 transformer layers, and a feed-forward size of 2,000. The context window was further extended to 285 tokens to better accommodate the

longer narrative sequences found in the Tiny Stories dataset. The model was initialized from previously saved pretrained weights (45m285T_pretrained.pth) rather than trained from scratch. The dataset of 500,000 stories was split into a training set of 490,000 samples (98%) and a validation set of 10,000 samples (2%), and training was conducted for 3 epochs with a batch size of 16. A base learning rate of 0.001 was used with a cosine decay schedule, 30% warm-up steps, and a minimum learning rate floor of $5e-5$. Dropout, weight decay and label smoothing were all set to 0.05. Unlike the Tiny Chat training, this experiment did not use mixed precision (bfloat16), with all computations running in full precision. Gradient clipping at norm 1.0 was applied at each update step. Validation loss and perplexity were computed at the end of each epoch to monitor generalization performance.

Story Generator Evaluation

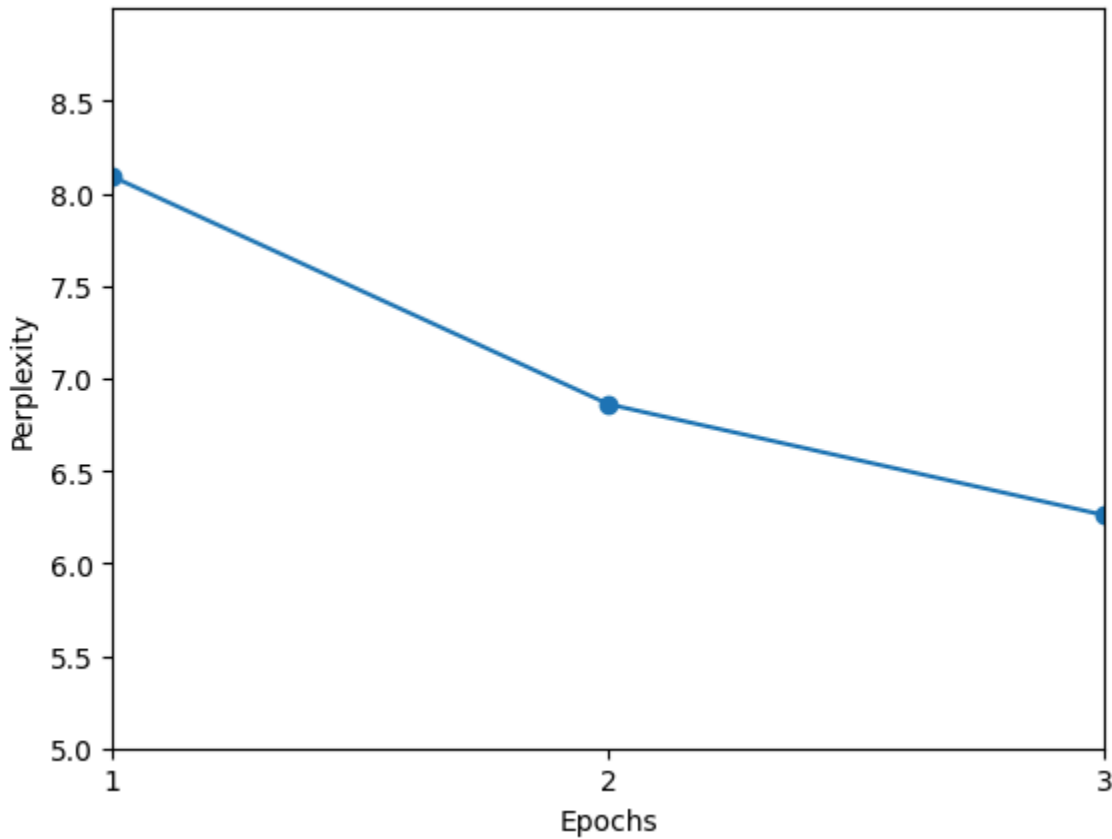
During training, both the training and validation loss decreased steadily, indicating that the model was learning effectively. The training loss started at 2.44 in the first epoch and decreased to 1.87 by the third epoch, while the validation loss improved from 2.09 to 1.83 over the same period. The close alignment between training and validation loss across epochs suggests stable learning without signs of overfitting. Regularization techniques, including dropout, weight decay, and label smoothing, were applied to support generalization and may have contributed to the slightly higher training loss compared to validation loss in early epochs. Overall, the

model demonstrated smooth and consistent learning throughout the three epochs.



Story Gen Train and Validation Loss Curve

Perplexity was calculated on the validation set to measure the model's uncertainty in predicting tokens. The results showed a consistent decline from 8.09 in the first epoch to 6.86 in the second, and finally 6.26 in the third epoch. This indicates that the model became progressively more confident and accurate in its token predictions, reflecting improved learning of general language patterns and alignment with the observed loss trends.



Story Gen Perplexity Curve

Story Gen Results

After training, the model was evaluated on custom prompts to assess its ability to generate short stories from a given starting sentence. The model successfully produced coherent narrative continuations, demonstrating an understanding of story structure and basic plot development as shown in the table below. For example, given the prompt “Once upon a time there was this planet,” it generated a story about a little girl named Lucy exploring a large planet, maintaining a logical sequence of events and

descriptive elements. Similarly, for the prompt “There was a storm,” the model produced a detailed account of a storm, including sensory descriptions of wind, rain, and ground movement.

While the model was able to maintain coherence and generate engaging content, it occasionally repeated phrases or sentences, as seen in the storm story, reflecting limitations in vocabulary and training size. Overall, the model demonstrates promising story generation capabilities with coherent narrative flow, though it shows room for improvement in reducing repetition and enhancing variety.

Prompt:
Once upon a time there was this planet

Story:
once upon a time there was this planet. it was very big and very big. one day, a little girl named lucy wanted to explore the planet. she asked her mom if she could go. her mom said yes, so lucy ran outside. she saw the big planet and it was very big. she wanted to explore it. so lucy started to climb the big planet. she climbed and climbed until she reached the top. she looked around and saw lots of things. she saw a big tree and some birds flying in the sky. she was so happy! lucy kept climbing until she reached the top. she looked around and saw the big planet. she was so excited! she wanted to explore it. so she climbed down the big planet and ran back home. she had a lot of fun exploring the big planet.

Prompt:
There was a storm

Story:
there was a storm. it was very loud and scary. the wind was blowing and the rain was pouring down. the wind was so strong that it made the rain drop down. the rain was so strong that it made the ground shake. the rain was so strong that it made the ground shake even harder. the rain kept falling and the ground shook even harder. the rain kept falling and the ground shook even harder. the rain kept falling and the ground shook even harder. the rain kept falling and the ground shook even harder. the rain kept falling and the ground shook until it was so heavy that it was almost like it was shaking. the rain stopped and the sun came out. the ground was still shaking, but the ground was still shaking. the storm was so strong that it made the ground shake even harder. the rain stopped and the ground was quiet again. the sun came out and the ground was dry again. the ground was still shaking. but the ground was still shaking.

Story Gen Test Prompts Table

Conclusion

In conclusion, this project demonstrates that it is possible to train small decoder-only transformer language models from scratch with limited data and computational resources, achieving coherent text generation across multiple tasks. Using a complete end-to-end PyTorch pipeline, three distinct models Tiny Chat, Casual Chat, and Story Generator were trained and evaluated on their respective datasets. Across experiments, training and validation losses showed smooth and stable learning, perplexity consistently decreased, and the models produced meaningful outputs on custom test prompts. While the models exhibit promising coherence and context understanding, limitations in size, dataset, and training duration occasionally led to repetition or less precise responses. Overall, the experiments highlight the effectiveness of the pipeline and model architectures for generating both conversational and narrative text, and provide a foundation for further improvements with larger models or extended training.

Inference

How To Use The Trained Models

1. Clone the github repository
2. Create and activate virtual environment:
 - `python -m venv .venv`
 - `source .venv/bin/activate`
3. Run inference script for one of the models

Casual Chat:

```
python Casual_chat_Inf.py
```

Tiny Chat:

```
python Tiny_Chat_Inf.py
```

Story Gen:

```
python Story_Gen_Inf.py
```

Additionally, Top_K and Temperature can be passed in the command line for each script.

Example: `python Casual_chat_Inf.py --Top_K 20 --Temp 0.60`

(double dash before the parameter name)

Datasets Credit

1. DailyDialog: A Manually Labelled Multi-turn Dialogue Dataset

License: DailyDialog dataset is licensed under CC BY-NC-SA 4.0.

Link:

<https://huggingface.co/datasets/roskoN/dailydialog>

2. General_Conversation_Mixed_Dataset

Link:

https://huggingface.co/datasets/cosmosai471/General_Conversation_Mixed_Dataset

3. HuggingFaceTB/everyday-conversations-llama3.1-2k

Description: This dataset contains 2.2k multi-turn conversations generated by Llama-3.1-70B-Instruct. We ask the LLM to generate a simple multi-turn conversation, with 3-4 short exchanges, between a User and an AI Assistant about a certain topic.

credit: @misc{everydayconversations2024,
author = {Hugging Face},
title = {Everyday Conversations for LLMs},
year = {2024},
howpublished =
 {url{<https://huggingface.co/datasets/HuggingFaceTB/everyday-conversations-llama3.1-2k>}}
}

Link:

<https://huggingface.co/datasets/HuggingFaceTB/everyday-conversations-llama3.1-2k>

4. Tiny Chat from HuggingFace

Description: This dataset comprises 1,000,000 synthetically generated short chat conversations, created using a specialized version of GPT-4o (referred to as GPT-4o mini). The conversations are primarily constructed using BASIC (British Academic Scientific International Commercial) English words and grammar. However, to ensure the coherence and fluidity of the dialogues, some non-BASIC English words have been included selectively.

Link: <https://huggingface.co/datasets/starhopp3r/TinyChat>

5. Tiny Stories from Huggingface

Description: Dataset containing synthetically generated (by GPT-3.5 and GPT-4) short stories that only use a small vocabulary.

Link: <https://huggingface.co/datasets/roneneldan/TinyStories>

7. Dataset for chatbot from kaggle

Link: <https://www.kaggle.com/datasets/grafstor/simple-dialogs-for-chatbot>

8. Custom samples

Description: hardcoded prompt and responses were created
samples are in Saved_Data/harcoded_samples.py